

(portada, índice...)

1. Introducción.

1. Introducción

En este documento se especifica el diseño de los protocolos de nivel de aplicación para **nanoFiles**. El sistema consiste en una plataforma de compartición y transferencia de ficheros que combina dos modelos de comunicación:

- **Modelo Cliente-Servidor:** Entre cada par (*peer*) y un servidor de directorio centralizado para la gestión del censo de usuarios y localización de ficheros.
- **Modelo Peer-to-Peer (P2P):** Entre los propios pares para la transferencia directa de los datos de los ficheros.

El objetivo es implementar una comunicación robusta utilizando tanto el protocolo **UDP** (para el directorio) como **TCP** (para la transferencia entre pares).

Mejoras consideradas

A la hora de diseñar los protocolos, hemos incluido las siguientes mejoras sobre la funcionalidad mínima para aumentar la robustez y capacidad del sistema:

- **Gestión de sesiones persistentes (Comando quit):** Se ha diseñado un mecanismo para mantener actualizado el censo del directorio en tiempo real. Cuando un par finaliza su ejecución, comunica su baja para que el resto de usuarios no intenten conectar con un servidor inactivo.
- **Protocolo de Directorio de gran capacidad (dirfiles ampliado):** El protocolo de comunicación con el directorio es capaz de gestionar listados de ficheros que superan el tamaño de un único datagrama UDP, permitiendo un intercambio de información fragmentado y fiable.
- **Descarga Multiorigen (Comando peerdl *):** Se ha diseñado el protocolo P2P para permitir la descarga de un mismo fichero desde múltiples pares simultáneamente, alternando los fragmentos para optimizar el ancho de banda.
- **Control de Integridad:** Se incluye el uso de *hashes* en los mensajes para identificar unívocamente los ficheros y asegurar que el receptor solicita el contenido exacto.

2. Formato de los mensajes del protocolo de comunicación con el Directorio

Para definir el protocolo de comunicación con el Directorio, utilizamos mensajes textuales con formato "**campo:valor**". El campo operation determina el tipo de mensaje y los campos que lo acompañan.

Tipos y descripción de los mensajes

Mensaje	Sentido	Descripción
login	Cliente → Dir	Solicita registrar un nickname en el censo.
login_ok	Dir → Cliente	Confirma que el registro se ha realizado con éxito.
get_peers	Cliente → Dir	Solicita la lista de peers activos (comando peers).
peer_list	Dir → Cliente	Contiene el listado de nicknames e IP:puerto de los peers.
register	Cliente → Dir	Registra al peer como servidor de ficheros (comando serve).
get_files	Cliente → Dir	Solicita la lista de ficheros disponibles en el directorio.
logout	Cliente → Dir	Notifica la salida del sistema para darse de baja (comando quit).

Ejemplo de registro de servidor (Comando serve):

operation:register

nickname:alicia

port:12345

3. Formato de los mensajes del protocolo de transferencia de ficheros

Para la comunicación entre pares (P2P), utilizamos mensajes **binarios multiformato** sobre TCP. El primer byte corresponde al opcode, que identifica la operación.

Tabla de Opcodes (según PeerMessageOps.java)

Operación	Opcode (hex/dec)	Descripción
Present	0x00 (0)	Mensaje de presentación inicial.
FileList	0x01 (1)	Envío de la lista de ficheros compartidos.
Download	0x02 (2)	Petición de descarga de un fichero por su hash.
FileData	0x03 (3)	Envío de los datos/bloques del fichero solicitado.
Files	0x04 (4)	Listado de metadatos de ficheros disponibles.
FileNotFound	0x05 (5)	Error: El fichero solicitado no existe en el peer.

Estructura del mensaje binario (Ejemplo Download):

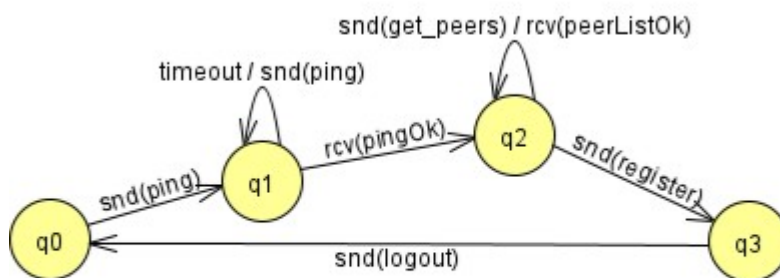
1. **Opcode** (1 byte): 0x02.
2. **Hash Length** (4 bytes): Tamaño de la cadena del hash.
3. **Hash Data** (Variable): Contenido del hash del fichero solicitado.

4 Autómatas de protocolo

Con respecto a los autómatas, hemos considerado las siguientes restricciones:

- Un cliente del directorio no puede pedir la lista de usuarios si no ha iniciado sesión previamente.
- ...
-

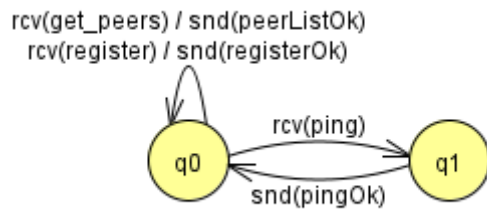
Autómata rol cliente de directorio



Estado q0 (Inicio): Estado inicial.

- **Transición q0 → q1:** Etiqueta: `snd(ping)`.
- **Transición q1 → q1 (Bucle):** Etiqueta: `timeout / snd(ping)`. Esto representa la confiabilidad sobre UDP.
- **Transición q1 → q2:** Etiqueta: `rcv(pingOk)`.
- **Transición q2 → q2 (Bucle):** Etiqueta: `snd(get_peers) / rcv(peerListOk)`. (Para el comando `peers`).
- **Transición q2 → q2 (Bucle):** Etiqueta: `snd(fileList) / rcv(fileListOk)`. (Para el comando `dirfiles`).
- **Transición q2 → q3:** Etiqueta: `snd(register)`. Ocurre al ejecutar el comando `serve`.
- **Transición q3 → q3 (Bucle):** Aquí puedes seguir consultando `peers` o ficheros mientras eres servidor.
- **Transición q3 → q0:** Etiqueta: `snd(logout)`. Ocurre al ejecutar `quit` para darte de baja.

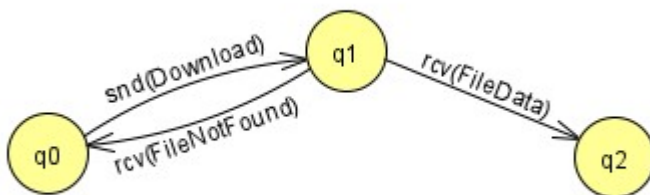
Autómata rol servidor de directorio



Es el que gestiona las peticiones en el puerto 6868. Es un autómata reactivo.

- **Estado q0 (Listening):** Estado inicial y final de cada operación.
- **Transición q0 → q1:** Etiqueta: `rcv(ping)`.
- **Estado q1 (Procesando):** El servidor verifica la compatibilidad.
- **Transición q1 → q0:** Etiqueta: `snd(pingOk)`.
- **Otras transiciones desde q0:** Crea bucles similares para `rcv(register)/snd(registerOk)` y `rcv(get_peers)/snd(peerListOk)`.

Autómata rol cliente de ficheros



Este rige la conexión directa entre pares usando los opcodes de `PeerMessageOps`.

- **Estado q0 (Idle):** Listo para iniciar una descarga.
- **Transición q0 → q1:** Etiqueta: `snd(Download)`. (Corresponde al Opcode 2 en tu código).
- **Estado q1 (Awaiting Data):** Esperando la respuesta del otro peer.
- **Transición q1 → q2 (Éxito):** Etiqueta: `rcv(FileData)`. (Opcode 3). El cliente recibe los bytes y cierra la conexión.

- **Transición q1 → q0 (Error):** Etiqueta: `rcv(FileNotFound)`. (Opcode 5). Se informa del error al usuario.

Autómata rol servidor de ficheros



Este se ejecuta en un hilo separado (NFServerThread).

- **Estado q0 (Listening):** Esperando conexiones entrantes en el puerto TCP registrado.
- **Transición q0 → q1:** Etiqueta: `rcv(Download)`.
- **Estado q1 (Searching):** El servidor busca el hash en su FileDatabase.
- **Transición q1 → q0 (Si existe):** Etiqueta: `snd(FileData)`. Envía el contenido del fichero.
- **Transición q1 → q0 (Si NO existe):** Etiqueta: `snd(FileNotFound)`. Envía el error y cierra el socket de esa conexión.

5. Ejemplo de intercambio de mensajes (Conversación)

A continuación, se muestra una interacción típica para el comando `peers`:

1 CLIENTE (envía solicitud):

operation: `get_peers`
nickname: `alumno1`

2 DIRECTORIO (procesa y responde):

operation: `peer_list`
peer_count: `2`
peer_1: `alicia,127.0.0.1:10001`
peer_2: `bob,127.0.0.1:10002`